

# 毕设记录

经过对比(batch=4/8/16, epochs=100/200/300/400), 发现当 batch\_size=8, epochs=400 时 RMSE 相对较小。

## 一. 实验结果

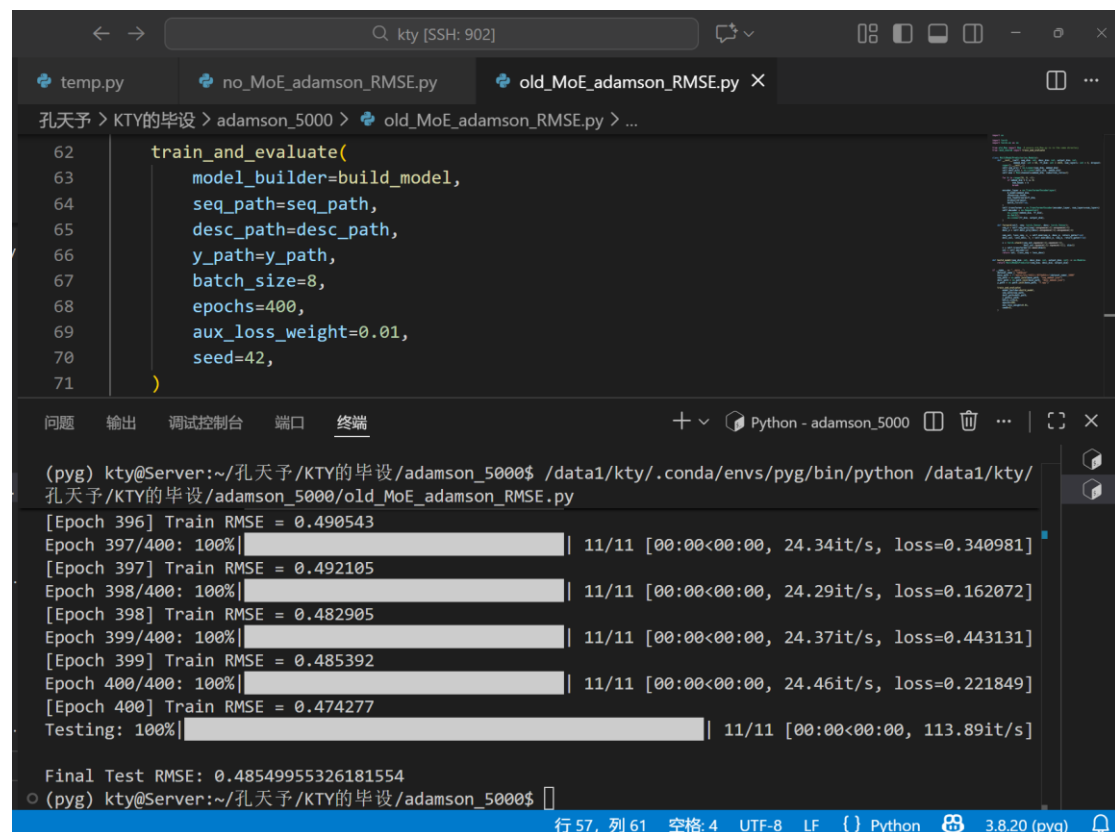
### (1) Adamson 数据集

1.无 MoE: RMSE=0.2581264433924334

在做 old\_MoE 的时候，是 stageA output -> moe -> transformer，属于流程的简单拼接，这样会导致 transformer 学到的东西严重失真，因此可以看到 old\_Moe\_{dataset} RMSE 甚至比不加 MoE 还要高，两个数据集都是如此。

叶师兄的 MoE 代码是图像结构，也就是适配 gan 的。在这个基础上做修改：把 input 展平了，且没有做激活函数。将 moe 融入 transformer，通过修改 transformer 架构而不是简单拼接，这样效果更好，而且改进点更加创新。

## 2. 初始 MoE: RMSE=0.48549955326181554



```
temp.py no_MoE_adamson_RMSE.py old_MoE_adamson_RMSE.py X
孔天子 > KTY的毕设 > adamson_5000 > old_MoE_adamson_RMSE.py > ...

62     train_and_evaluate(
63         model_builder=build_model,
64         seq_path=seq_path,
65         desc_path=desc_path,
66         y_path=y_path,
67         batch_size=8,
68         epochs=400,
69         aux_loss_weight=0.01,
70         seed=42,
71     )

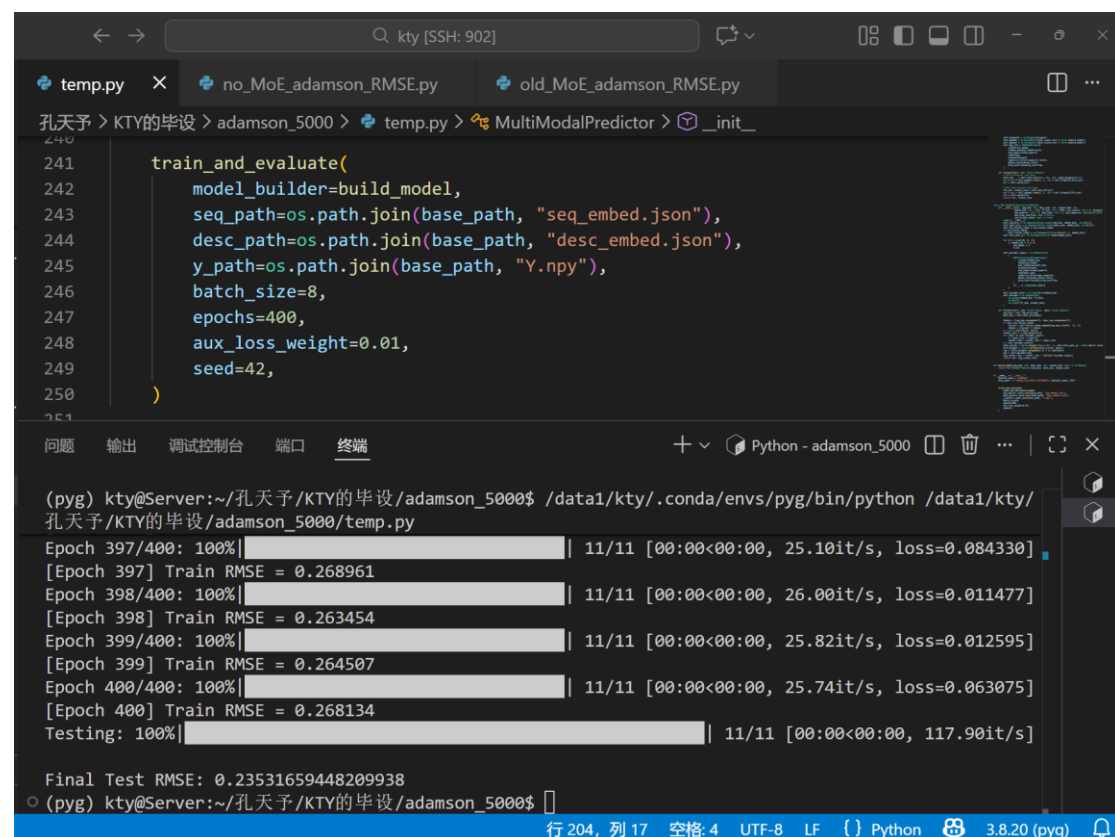
问题 输出 调试控制台 端口 终端
Python - adamson_5000

(pyg) kty@Server:~/孔天子/KTY的毕设/adamson_5000$ /data1/kty/.conda/envs/pyg/bin/python /data1/kty/
孔天子/KTY的毕设/adamson_5000/old_MoE_adamson_RMSE.py

[Epoch 396] Train RMSE = 0.490543
Epoch 397/400: 100%|████████████████████| 11/11 [00:00<00:00, 24.34it/s, loss=0.340981]
[Epoch 397] Train RMSE = 0.492105
Epoch 398/400: 100%|████████████████████| 11/11 [00:00<00:00, 24.29it/s, loss=0.162072]
[Epoch 398] Train RMSE = 0.482905
Epoch 399/400: 100%|████████████████████| 11/11 [00:00<00:00, 24.37it/s, loss=0.443131]
[Epoch 399] Train RMSE = 0.485392
Epoch 400/400: 100%|████████████████████| 11/11 [00:00<00:00, 24.46it/s, loss=0.221849]
[Epoch 400] Train RMSE = 0.474277
Testing: 100%|████████████████████| 11/11 [00:00<00:00, 113.89it/s]

Final Test RMSE: 0.48549955326181554
(pyg) kty@Server:~/孔天子/KTY的毕设/adamson_5000$
```

## 3. 融合 Transformer 之后: RMSE=0.23531659448209938



```
temp.py X no_MoE_adamson_RMSE.py old_MoE_adamson_RMSE.py
孔天子 > KTY的毕设 > adamson_5000 > temp.py > MultiModalPredictor > __init__

241     train_and_evaluate(
242         model_builder=build_model,
243         seq_path=os.path.join(base_path, "seq_embed.json"),
244         desc_path=os.path.join(base_path, "desc_embed.json"),
245         y_path=os.path.join(base_path, "Y.npy"),
246         batch_size=8,
247         epochs=400,
248         aux_loss_weight=0.01,
249         seed=42,
250     )

问题 输出 调试控制台 端口 终端
Python - adamson_5000

(pyg) kty@Server:~/孔天子/KTY的毕设/adamson_5000$ /data1/kty/.conda/envs/pyg/bin/python /data1/kty/
孔天子/KTY的毕设/adamson_5000/temp.py

Epoch 397/400: 100%|████████████████████| 11/11 [00:00<00:00, 25.10it/s, loss=0.084330]
[Epoch 397] Train RMSE = 0.268961
Epoch 398/400: 100%|████████████████████| 11/11 [00:00<00:00, 26.00it/s, loss=0.011477]
[Epoch 398] Train RMSE = 0.263454
Epoch 399/400: 100%|████████████████████| 11/11 [00:00<00:00, 25.82it/s, loss=0.012595]
[Epoch 399] Train RMSE = 0.264507
Epoch 400/400: 100%|████████████████████| 11/11 [00:00<00:00, 25.74it/s, loss=0.063075]
[Epoch 400] Train RMSE = 0.268134
Testing: 100%|████████████████████| 11/11 [00:00<00:00, 117.90it/s]

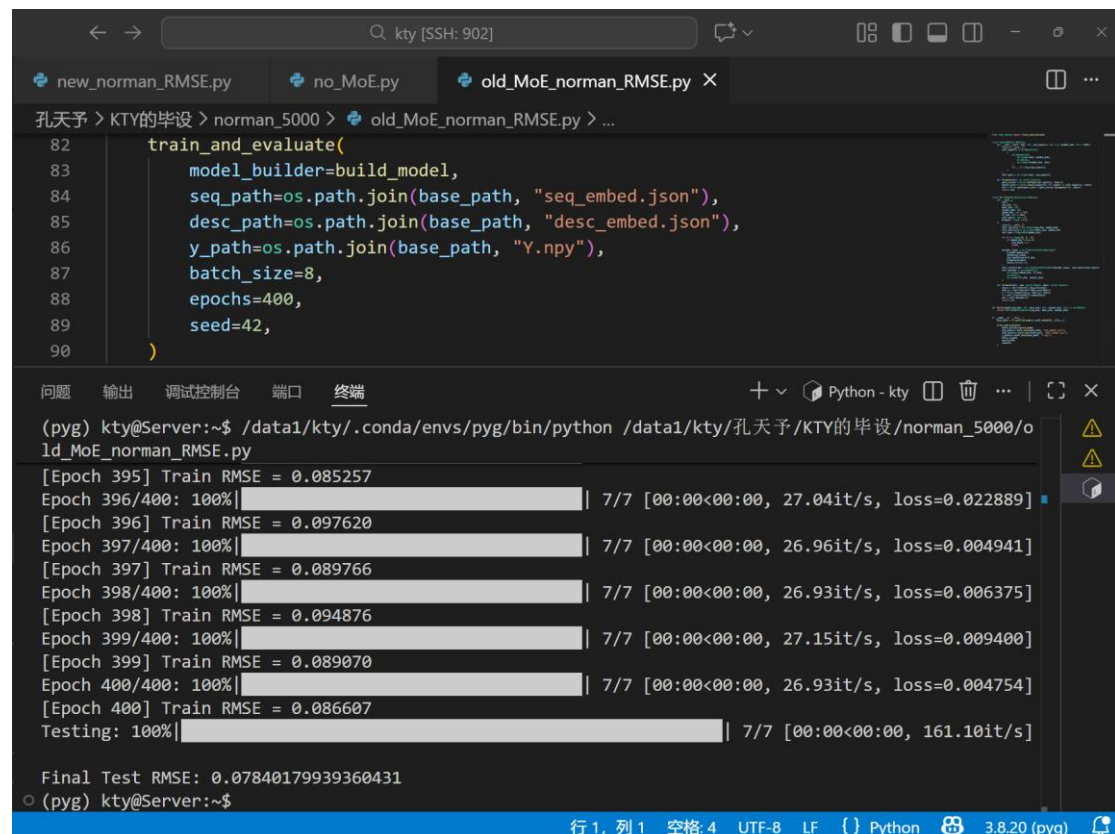
Final Test RMSE: 0.23531659448209938
(pyg) kty@Server:~/孔天子/KTY的毕设/adamson_5000$
```

4.进一步优化 Top-P（动态 K）:RMSE=0.20872576033174686

## (2) Norman 数据集

1.无 MoE: RMSE=0.07061933747943651

## 2. 初始 MoE: RMSE=0.07840179939360431

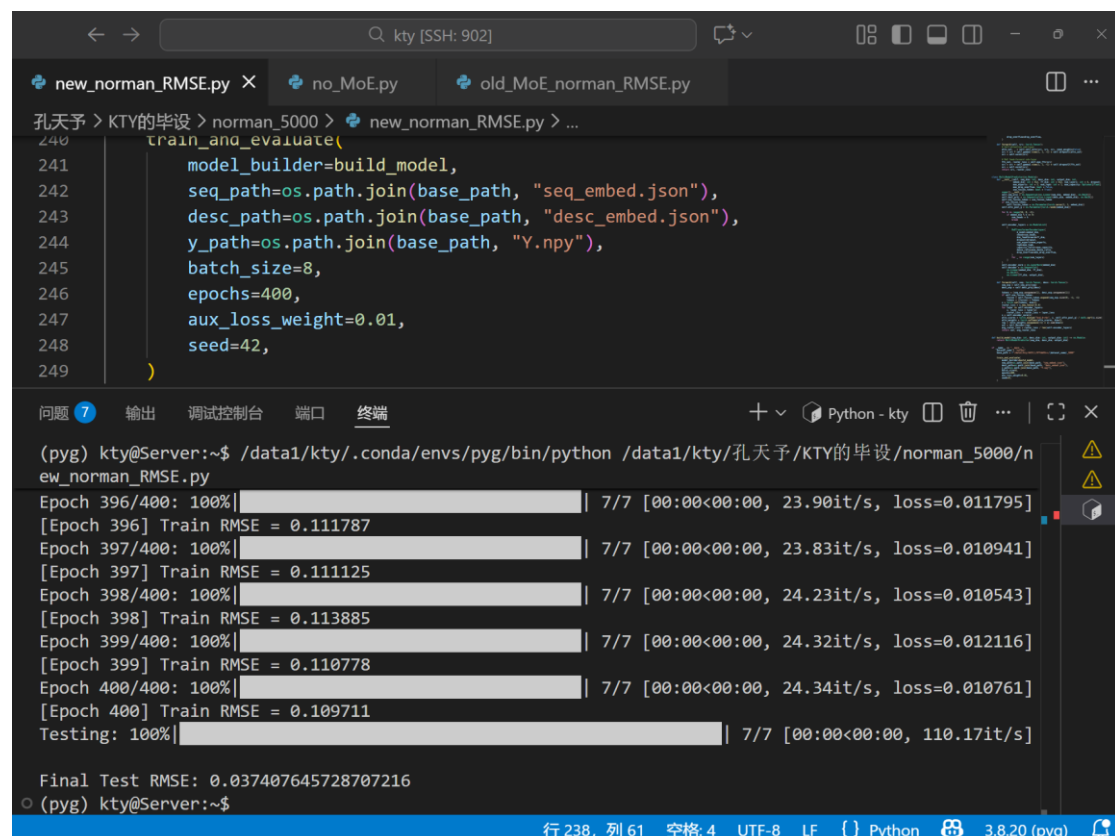


```
new_norman_RMSE.py no_MoE.py old_MoE_norman_RMSE.py X
孔天予 > KTY的毕设 > norman_5000 > old_MoE_norman_RMSE.py > ...
82     train_and_evaluate(
83         model_builder=build_model,
84         seq_path=os.path.join(base_path, "seq_embed.json"),
85         desc_path=os.path.join(base_path, "desc_embed.json"),
86         y_path=os.path.join(base_path, "Y.npy"),
87         batch_size=8,
88         epochs=400,
89         seed=42,
90     )

问题 输出 调试控制台 端口 终端
(pyg) kty@Server:~$ /data1/kty/.conda/envs/pyg/bin/python /data1/kty/孔天予/KTY的毕设/norman_5000/old_MoE_norman_RMSE.py
[Epoch 395] Train RMSE = 0.085257
Epoch 396/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 27.04it/s, loss=0.022889]
[Epoch 396] Train RMSE = 0.097620
Epoch 397/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 26.96it/s, loss=0.004941]
[Epoch 397] Train RMSE = 0.089766
Epoch 398/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 26.93it/s, loss=0.006375]
[Epoch 398] Train RMSE = 0.094876
Epoch 399/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 27.15it/s, loss=0.009400]
[Epoch 399] Train RMSE = 0.089070
Epoch 400/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 26.93it/s, loss=0.004754]
[Epoch 400] Train RMSE = 0.086607
Testing: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 161.10it/s]

Final Test RMSE: 0.07840179939360431
(pyg) kty@Server:~$
```

## 3. 融合 Transformer 之后: RMSE=0.037407645728707216



```
new_norman_RMSE.py X no_MoE.py old_MoE_norman_RMSE.py
孔天予 > KTY的毕设 > norman_5000 > new_norman_RMSE.py > ...
240     train_and_evaluate(
241         model_builder=build_model,
242         seq_path=os.path.join(base_path, "seq_embed.json"),
243         desc_path=os.path.join(base_path, "desc_embed.json"),
244         y_path=os.path.join(base_path, "Y.npy"),
245         batch_size=8,
246         epochs=400,
247         aux_loss_weight=0.01,
248         seed=42,
249     )

问题 7 输出 调试控制台 端口 终端
(pyg) kty@Server:~$ /data1/kty/.conda/envs/pyg/bin/python /data1/kty/孔天予/KTY的毕设/norman_5000/new_norman_RMSE.py
Epoch 396/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 23.90it/s, loss=0.011795]
[Epoch 396] Train RMSE = 0.111787
Epoch 397/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 23.83it/s, loss=0.010941]
[Epoch 397] Train RMSE = 0.111125
Epoch 398/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 24.23it/s, loss=0.010543]
[Epoch 398] Train RMSE = 0.113885
Epoch 399/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 24.32it/s, loss=0.012116]
[Epoch 399] Train RMSE = 0.110778
Epoch 400/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 24.34it/s, loss=0.010761]
[Epoch 400] Train RMSE = 0.109711
Testing: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 110.17it/s]

Final Test RMSE: 0.037407645728707216
(pyg) kty@Server:~$
```

#### 4. 进一步优化 Top-P（动态 K）:RMSE=0.20872576033174686

```
temp.py
孔天子 > KTY的毕设 > norman_5000 > temp.py > ...
276 epochs=400,
277 aux_loss_weight=None, # ★ 不再使用单一 aux loss
278 seed=42,
279 )
...
问题 输出 调试控制台 端口 终端
(pyg) kty@Server:~$ /data1/kty/.conda/envs/pyg/bin/python /data1/kty/孔天子/KTY的毕设/norman_5000/temp.py
Epoch 393/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 22.74it/s, loss=0.000564]
[Epoch 393] Train RMSE = 0.037075
Epoch 394/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 23.00it/s, loss=0.000686]
[Epoch 394] Train RMSE = 0.046186
Epoch 395/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 22.73it/s, loss=0.002045]
[Epoch 395] Train RMSE = 0.038672
Epoch 396/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 22.60it/s, loss=0.002087]
[Epoch 396] Train RMSE = 0.041868
Epoch 397/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 22.72it/s, loss=0.001874]
[Epoch 397] Train RMSE = 0.045490
Epoch 398/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 22.27it/s, loss=0.000593]
[Epoch 398] Train RMSE = 0.044325
Epoch 399/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 22.60it/s, loss=0.001260]
[Epoch 399] Train RMSE = 0.041946
Epoch 400/400: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 22.82it/s, loss=0.000509]
[Epoch 400] Train RMSE = 0.042884
Testing: 100%|████████████████████████████████████████| 7/7 [00:00<00:00, 85.90it/s]
Final Test RMSE: 0.030404951168304793
(pyg) kty@Server:~$
```

## 二. MoE 改动说明

### 1. 原始 MoE 结构解析:

#### 稀疏 MoE:

- 每个样本只激活少量专家，节省计算

#### 多模态注意力专家:

- SelfAttentionCat / SelfAttentionAdd / CrossAttention
- 捕捉输入特征间的全局交互

#### 下采样 + 上采样:

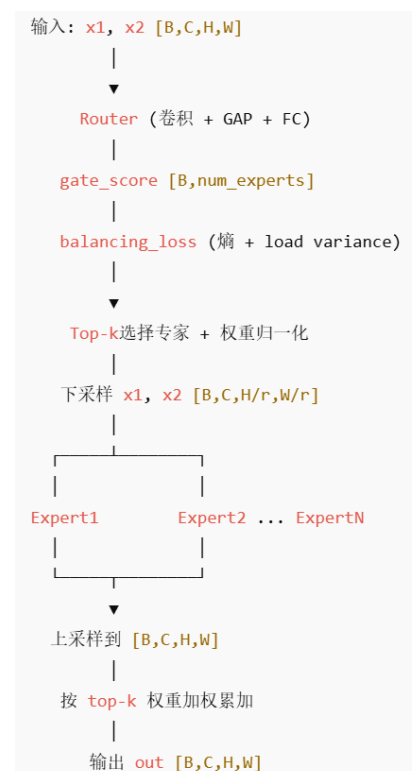
- 减少空间维度计算量

#### Router balancing loss:

- 确保专家均衡使用

#### Top-k 权重加权:

- 不同专家对样本贡献可加权混合





## 2. MoE 结合 Transformer:

与原始的 Moe.py 相比，针对 adamson 和 norman 数据集的改进版 MoE 代码在结构设计、路由机制和应用场景上有改动，主要创新点如下：

### (1) . 应用场景与输入形式的差异

#### • 原始 Moe.py:

针对处理二维图像特征（输入为[B, C, H, W]），专家网络（如 SelfAttentionCat、CrossAttention 等）主要用于融合两个图像特征图（x1 和 x2），通过下采样减小空间尺寸以提升效率，最终输出融合后的特征图。

#### • 改进版代码:

针对序列数据（文本或嵌入向量）设计，输入为 [B, S, C]（B 为批次，S 为序列长度，C 为通道数），用于多模态预测（如序列特征 seq 和描述特征 desc 的融合），输出为预测结果（如回归任务的输出值）。

### (2) . 路由机制（Router）的优化

#### • 原始 Moe.py:

路由网络基于卷积实现：通过两层卷积 + 全局池化提取特征，再用全连接层输出专家权重，适用于图像特征的全局信息提取。

平衡损失函数：结合熵损失（鼓励均匀使用专家）和负载方差损失（减少专家负载差异）。

#### • 改进版代码:

路由网络基于全连接层：更适合序列数据的 token 级路由（每个 token 独立计算专家权重），并引入温度系数（temperature）控制 softmax 分布的陡峭程度，增强路由的区分度。

平衡损失函数：简化为计算专家重要性的平方和（load\_balance\_loss），通过最小化该值鼓励专家负载更均衡，实现更高效的负载分配。

### (3) . 专家网络（Experts）的重构

#### • 原始 Moe.py:

专家网络以注意力机制为核心（如自注意力、交叉注意力），针对图像特征的空间关联建模，输入为两个特征图，输出融合后的特征图。

#### • 改进版代码:

专家网络替换为前馈网络（FeedForwardExpert），由线性层 + GELU 激活组成，更适合处理序列 token 的非线性变换，与 Transformer 架构中的 FFN 模块兼容，且计算效率更高。

(4) . 路由策略与容量控制

• 原始 Moe.py:

采用 top-k 路由（如每次选择 2 个专家），对每个样本的特征图整体路由，所有选中样本均由对应专家处理，无容量限制。

• 改进版代码:

采用 Switch-style 路由：限制 top-k=1（每次仅选 1 个专家），简化计算并降低冗余。

引入容量控制（capacity\_factor）：为每个专家设置最大处理 token 数，超出容量的 token 由一个轻量级稠密网络（dense\_ffn）处理，避免个别专家负载过高导致的训练不稳定，这是对工程化部署的重要优化。

(5) . 与 Transformer 架构的融合

• 原始 Moe.py:

独立的 MoE 模块，专注于特征图融合，未与 Transformer 结合。

• 改进版代码:

将 MoE 集成到 Transformer 编码器层（MoETransformerEncoderLayer），替换传统的 FFN 模块，形成 MoE-Transformer 混合架构。同时加入 layer scale（gamma1、gamma2）和 LayerNorm 等稳定训练的机制，更适合处理长序列数据。

(6) . 多模态融合设计

改进版代码:

新增 MultiModalPredictor 类，专门用于融合序列特征（seq）和描述特征（desc），通过可学习的 fusion\_token 增强模态间交互，并使用注意力池化（attn\_pool\_q）聚合序列信息，最终输出预测结果。这一设计针对多模态任务进行了定制，而原始代码未涉及多模态场景。

3. 结合论文:

| 项目       | Top-1 (你现在)  | Dynamic Routing (论文)        |
|----------|--------------|-----------------------------|
| 激活专家数    | 固定 1         | 动态 (1 ~ N)                  |
| 选择规则     | argmax       | 累积概率 $\geq p$               |
| $g(x)$   | softmax 归一   | 直接用 $P_i$ (不再归一)            |
| 额外损失     | Load balance | Load balance + Entropy loss |
| capacity | 可选           | 论文中 不强调 capacity            |

### 三. 代码解析

#### (1) Preprocess.py

##### 【输入】

1. 单细胞数据文件 (.h5ad):

```
adata = sc.read_h5ad("/data1/kty/孔天予  
/GRAPEmain/data/adamson/perturb_processed.h5ad")
```

- adata.obs['condition'] 包含 perturbation 条件
- adata.var['gene\_name'] 包含基因名

2. 基因 ID 映射文件 (.tsv):

```
"/data/genes_"+dataset_name+".tsv"
```

- 用于把 gene\_symbol 转换为 Ensembl ID
- 包含两列: gene\_id 和 gene\_symbol

3. 用户参数:

- dataset\_name: 如 "adamson" 或 "norman"
- gene\_num: 选择的目标基因数量 (top n HVG + perturbation 基因)

##### 【输出】

| 文件                     | 内容                                  | 用途                      |
|------------------------|-------------------------------------|-------------------------|
| perturb_gene_info.json | perturbation 基因的注释信息                | 后续模型输入/分析 perturbation  |
| perturb_gene_seq.json  | perturbation 基因序列                   | 可以用于序列嵌入/Transformer 输入 |
| perturb_miss.json      | 查询失败的 perturbation 基因               | 调试/检查                   |
| target_gene_info.json  | 所有目标基因注释信息 (top HVG + perturbation) | 模型训练特征                  |
| target_gene_seq.json   | 所有目标基因序列                            | 作为 Transformer/语言模型输入   |
| target_miss.json       | 查询失败的目标基因                           | 调试                      |

##### 【功能】

解析 .h5ad 数据集中的 perturbation 条件

查询并保存基因注释和序列

选取 top gene\_num 高变基因 (保留 perturbation genes)

输出 JSON 文件供后续模型训练使用



## (2) extract\_embed.py

### 【输入】

#### 1. 基因信息 JSON 文件

info\_path="/data1/kty/孔天予/result/norman\_5000/target\_gene\_info.json"

- 来自前一步的 preprocess 脚本
- 包含每个基因的描述（description）和其他信息

#### 2. 基因序列 JSON 文件

seq\_path="/data1/kty/孔天予/result/norman\_5000/target\_gene\_seq.json"

- 来自前一步的 preprocess 脚本
  - 包含每个基因的 DNA 序列
- #### 3. 预训练模型 checkpoint
- DNA 模型: from\_pertrain\_DNA\_hyena
  - BERT 模型: from\_pertrain\_bert

#### 4. 参数

- max\_length: DNA 片段最大长度（8192）
- dataset\_name: 选择数据集（norman 或 adamson）
- hvg\_num: 选择的目标基因数量
- 

### 【输出】

| 文件              | 内容          | 说明                      |
|-----------------|-------------|-------------------------|
| desc_embed.json | 基因描述文本嵌入    | 维度 [hidden_size]，作为描述特征 |
| seq_embed.json  | 基因 DNA 序列嵌入 | 维度 [hidden_size]，作为序列特征 |

### 【功能】

将基因描述文本编码为向量

将基因 DNA 序列编码为向量（支持长序列切片）

保存嵌入到 JSON 文件，为后续训练使用

### (3) preprocess.py

原始 AnnData → 筛选高变异基因 → HVG 基因名列表 genes.json

AnnData 中 ctrl 细胞 → 平均表达 → ctrl.npy

AnnData 中每个扰动条件 → 平均表达 → Y.npy

条件字符串解析为基因名 → 对应 HVG 索引 → cond2gene\_idx.npy

条件名称统计 → conditions.json

| 文件名               | 内容           | 形状 / 说明                                |
|-------------------|--------------|----------------------------------------|
| ctrl.npy          | 控制组平均表达      | [num_genes]                            |
| Y.npy             | 每个扰动条件的平均表达  | [num_conditions (不含 ctrl) x num_genes] |
| cond2gene_idx.npy | 条件对应 KO 基因索引 | 列表数组, 每个元素是条件对应的基因索引 (支持多基因 KO)        |
| genes.json        | 基因名列表        | 长度 = target_gene_num                   |
| conditions.json   | 条件名称列表       | 包含 ctrl 和所有扰动条件                        |

### (4) RMSE.py

#### 【功能】

核心功能:

1. 加载 Norman 数据集的基因序列嵌入 (seq\_embed.json) 和 基因描述嵌入 (desc\_embed.json)
2. 加载每个条件的平均表达矩阵 Y.npy
3. 对序列和描述嵌入进行 标准化
4. 构建 多模态预测模型:
  - MoE (Mixture-of-Experts) 层分别处理序列和描述嵌入
  - Transformer 将序列与描述信息融合
  - 最后通过 MLP 解码为预测表达值
5. 使用 MSELoss 训练模型, 并加上 router load loss 作为正则
6. 训练完成后在训练集上进行 预测和 RMSE 计算

模型特性:

- 支持多专家路由 (MoE)
- 支持多模态输入 (序列 + 描述)
- Transformer 融合序列与描述信息
- 输出为每个条件对应的平均表达预测

【输入】

| 输入文件            | 类型          | 内容                                      |
|-----------------|-------------|-----------------------------------------|
| seq_embed.json  | JSON        | 每个基因对应的 DNA 序列嵌入 (num_genes, embed_dim) |
| desc_embed.json | JSON        | 每个基因对应的描述嵌入 (num_genes, embed_dim)      |
| Y.npy           | NumPy array | 每个条件的平均表达 (num_conditions, num_genes)   |

- 脚本参数:
  - batch\_size：训练批大小
  - epochs：训练轮数
  - lr：学习率

【输出】

| 输出        | 类型           | 内容                                       |
|-----------|--------------|------------------------------------------|
| model     | PyTorch 模型对象 | 训练好的 MultiModalPredictor 模型              |
| rmse      | float        | 在训练集（或测试集，因为这里使用相同数据）上的最终 RMSE           |
| all_preds | Tensor       | 模型对每个条件的预测表达 (num_conditions, num_genes) |

中间计算：

- router\_loss：MoE 层的负载均衡损失，用于训练正则
- epoch\_loss：每轮训练的 MSE + router loss
- train\_rmse：每轮训练 RMSE